



Lasso 9

Language Guide

- Overview
 - Programming Fundamentals
 - Web Applications
 - Database Interaction
 - Data Types
 - Defining
 - Defining Methods ▶
 - Defining Types
 - Defining Traits
 - On-demand Libraries
 - Programming Concepts
 - Backward Compatibility
 - Upgrading
 - Extending Lasso with C
 - Extending Lasso with Java
- Appendix

Legal Notices

Chapter 6.1

Defining Methods

This chapter provides details about how methods are defined and called in Lasso 9. Topics include methods, signatures, parameter types and multiple dispatch.

- ...

Introduction

Methods are the fundamental process abstraction in many languages, including Lasso 9. Methods provide a means for encapsulating a series of expressions so that they can be called repeatedly as a group. Complex, multi-step tasks are best expressed as a group of related methods. A method is defined under a specific name and is associated with a signature and a code block.

This chapter provides in-depth information about how methods are defined in Lasso 9.

Signatures

Before method definitions can be understood, it is important to understand signatures. A signature is a description of a method and includes its name, parameter names and types, and the method's return value type. Signatures are used when defining methods, and simplified signatures are used when defining types and traits. This chapter will concentrate on signatures for defining methods only.

Method names in Lasso consist of letters, numbers, and the underscore `_` character. A method name must start with a letter or one or two leading underscores followed by a letter. Lasso does not take case into account when comparing method names.

Method names beginning with an underscore are generally intended to be used internally only, and represent methods that might change in the future and are thus considered unstable.

Some valid examples of method names are shown below.

```
Field
_date_msec
Encode_Base64
String_ReplaceRegexp
```

There are several other characters which are valid in specific circumstances. The mathematical symbols `+` `-` `*` `/` `%` = are used in method names when supplying implementations for these operations in data types. See the *Defining Types* chapter for more information.

Most signatures consist of a method name followed by parentheses which surround a list of parameters for the method, and an optional return type.

A signature for the `loop` method is shown below. The parameter list includes three parameters: a `'to'` parameter which must be an integer and two more integer parameters each with default values of 1.

```
loop(to::integer, from::integer=1, by::integer=1)
```

When a method is called, the parameter names given in the method's signature become the variable names for those parameters within the method's body. The `loop` method above would have access to the local variables `#to`, `#from` and `#by`.

A signature's parameter list allows the specification of required and optional parameters, type constraints, keyword parameters, and rest parameters.

Empty Signature

A signature which is specified with an empty set of parentheses indicates that the method will not require or accept any parameters. Giving parameters to a method defined with such a signature with result in a failure.

```
method_name( )
```

Rest-Only Signature

A signature which is specified with an ellipsis `...` in parentheses indicates that the method will accept any parameters. A method defined with such a signature can be called with no parameters or any combination of values and keywords.

```
method_name(...)
```

Contents

- Chapter 6.1
- Defining Methods
- Introduction
- Signatures
 - Empty Signature
 - Rest-Only Signature
 - Required Parameters
 - Optional Parameters
 - Mixing Required and Optional Parameters
 - Keyword Parameters
 - Boolean Keywords
 - Rest Parameters
 - Type Constraints
 - Return Type
 - Type Binding
 - Syntax Diagram
- Defining Methods
 - Methods Returning Simple Expressions
 - Code Blocks
 - Syntax Diagram
- Multiple Dispatch
 - The Dispatch Process
 - Using Multiple Dispatch
 - Example: Constraints
 - Example: Arity

Note that the ellipsis should be specified as three periods rather than as a Unicode ellipsis character.

Within the method, the parameters which were passed in can be accessed through the `#rest` variable. If any parameters were given, the `#rest` variable will be of type `staticarray`. If no parameters are given, the `#rest` variable will be of type `VOID`. This signature can be useful for methods such as `inline` which have highly variable parameters that need special processing.

Required Parameters

Required parameters can be specified within a signature by naming them in order. All required parameters must be listed before any other parameter types. When calling a method with required parameters, all parameter values must be provided in the proper order according to the method's signature.

The name of each required parameter must be a valid variable name. Each name should consist of only letters, numbers, or underscore `_` and should start with a letter or underscore and a letter.

The following signature would define two required parameters named `firstname` and `lastname`. Within the method these parameters can be accessed through the local variables `#firstname` and `#lastname`.

```
method_name(firstname, lastname)
```

When calling this method, both parameters must be given in order.

```
method_name('Henry', 'Gibbons')
```

The parameter names are only used within the method so the particular parameter names need only make sense to the implementer of the method. However, the parameter names may be used in documentation or reported in error messages so they should be made descriptive when possible. Knowing a method requires the parameter `firstname` is more descriptive than a method which requires the parameter `fn`.

Optional Parameters

Optional parameters are those which are listed in a method's signature but are not required to be given values when the method is called. Optional parameters are specified within a signature by providing default values along with the parameter names. A default value is specified after a parameter name by using an equal sign `=` followed by an expression. The expression's value will be used to assign the default value to the parameter's local variable, should that value be omitted by the caller.

The default value expression will be evaluated independently each time as required. The default value expression will be evaluated from within the associated method's body, so any state valid at the beginning of the associated method is valid during the evaluation of all optional parameter default values.

Although optional parameters may be omitted when calling a method, when optional parameter values are provided, they must be provided in-order. That is, when the method is called, once an optional parameter is omitted, all subsequent optional parameters must also be omitted.

The parameters in the following signature are both optional. If the `host` parameter is not specified the variable `#host` within the method will have the default value `'localhost'`. If the `port` parameter is not specified, it will have the default value of 80.

```
connect(host='localhost', port=80)
```

When the method is called the parameters which are passed to it will be assigned to each of the optional parameters in turn. The method called as `connect('www.lassosoft.com')` will have a default port value of 80. The method called as `connect()` will have both default values. And, the method called as `connect('www.lassosoft.com', 443)` will use the specified values, overriding both defaults.

Mixing Required and Optional Parameters

When calling a method which accepts both required and optional parameters, all required parameter values must be specified before any optional parameter values. The values which are passed will be assigned to the required parameters first. While there are sufficient remaining values, the optional parameters will assigned in-order.

For example, the following signature has one required parameter `host` and two optional parameters: `port` and `timeout`.

```
connect(host, port=80, timeout=15)
```

`host` must be provided before `port` can be provided, and both `host` and `port` must be provided before `timeout` can be provided.

Keyword Parameters

Keyword parameters are named parameters which can be specified in any order. When keyword parameter values are passed to a method, they are given with the associated

parameter name, using the following syntax.

```
-parameterName = expression
```

If a method has any required or optional parameters they must be specified before the keyword parameters in both the method signature and when calling the method.

Keyword parameters are specified by preceding the parameter name with a hyphen -. Within the method body, the keyword parameter's associated local variable will not have the hyphen. Keyword parameters can be either required or optional. Optional keyword parameters are indicated in the same manner as regular optional parameters, by following the parameter name with an equals = and a default value.

For example, a hypothetical `find_in_string` method might have the following signature. The required input is followed by two keywords: the required `-find` and the optional `-ignoreCase`.

```
find_in_string(input, -find::string,
-ignoreCase::boolean=false)
```

When this method is called the input must always be given first. However, the two keywords can be given in either order, provided they follow all non-keyword parameters. It is valid to call the method in all of the following ways.

```
find_in_string('the fox', -find='x', -ignoreCase=true)
find_in_string('the fox', -ignoreCase=true, -find='x')
find_in_string('the fox', -find='x')
```

Within the method's body, three pre-defined local variables will be created for these parameters including `#input`, `#find` and `#ignorecase`.

Note that calling the method as `find_in_string('the fox')` will generate a failure since the `-find` keyword parameter is required (since it has no default value). Calling the method as `find_in_string(-find='x', 'the fox')` will also generate a failure since the input is being specified after a keyword. All required or optional parameters must be specified before the first keyword parameter.

Boolean Keywords

Often, keyword parameters will be used to indicate simple boolean values. For example, as a set of options or flags given to a method to control the details of its behavior. When calling a method, a keyword parameter can be passed without an associated value. Doing so is implicitly the same as passing a boolean `true` value for that parameter. Boolean keywords are normally specified with a default value of `false` so if the keyword is not specified the pre-defined variable will have a value of `false`.

The following signature defines the method `server_date` as accepting either a `-short` or a `-long` keyword parameter or neither one.

```
server_date(-short=false, -long=false)
```

If the method is called as `server_date(-short)` then the pre-defined variable `#short` will have a value of `true` and the pre-defined variable `#long` will have a value of `false`. If the method is called as `server_date()` then both variables will have a value of `false`.

Rest Parameters

The list of parameters may end with an ellipsis ... in order to specify that the method should accept any specified required and optional parameters as well as a variable number of additional parameters. The additional parameters are known as Rest parameters. When the method is called, any additional parameters are placed into a pre-defined `#rest` variable. If there are no rest parameter, the `#rest` variable will have a type of `VOID`. Otherwise, `#rest` will be a `staticarray`, holding the rest of the parameter values.

The signature below specifies that the `string_concatenate` method requires one parameter named `value`, but will accept any number of additional parameters. Within the method, the first parameter will be placed into the pre-defined variable `#value` and the remaining parameters, if any, will be placed into the pre-defined variable `#rest`.

```
string_concatenate(value, ...)
```

Note that the ellipsis should be specified as three periods rather than as a Unicode ellipsis character.

By default, the rest variable is always named `#rest`, but an alternate rest variable name can be specified in the signature by placing the desired name immediately after the ellipsis. The following signature would rename the rest variable to `#other`.

```
string_concatenate(value, ...other)
```

Type Constraints

In a signature, all parameter types, with the exception of the rest parameter, can be specified with an optional type constraint. While parameter count and ordering insure that the caller is passing the right number of parameters in the right order, type constraints insure that the parameter values are of the right type. For example, a method that expects to receive two string parameters, if given two integers, is being used incorrectly. If a caller

passes a parameter value which does not fit the type constraint set for that parameter, then a failure will be generated. Any type or trait name can be used as a constraint, and all parameter values must pass the `isa` test for their constraint before the method body begins to execute. Additionally, all parameter default values must produce results of a type which fits the type constraint set for their respective parameters.

A type constraint is specified by following the parameter name with a double colon `::` followed by a type name. Whitespace is permitted on either side of the `::` operator (examples herein will not include whitespace). The signature below has both of its required parameters constrained to only accept values which are of type `string`.

```
method_name(firstname::string, lastname::string)
```

If the parameter has a default value, it should be placed after the type constraint.

```
method_name(firstname::string, lastname::string = '<not provided>')
```

A parameter with no type constraint will accept any type of value. Constrained and unconstrained parameters can be mixed.

```
method_name(firstname::string, lastname)
method_name(firstname, lastname::string)
method_name(firstname::string, lastname::string,
            -age::decimal=0.0,
            -dept='')
```

Return Type

Specifying a return type for a signature enforces that the value returned by the associated method is of a specific type. If a method returns a value having a type which does not pass the `isa` test for the specified return type, then a failure is generated. Specifying a return type provides knowledge to the caller of the method about the method's result value. It also insures to the method's developer that their programming is correct, at least with respect to that method returning the proper value type. Specifying a return type is optional, and a method without a specified return type may return values of any type, or may return no value at all (in which case the value returned to the caller is `VOID`).

The return type for a signature is specified at the end of the signature, following the parameter list parentheses, by including two colons `::` followed by a type or trait name.

The following signature specifies that the method will always return a string value.

```
string_concatenate(value, ...other)::string
```

Type Binding

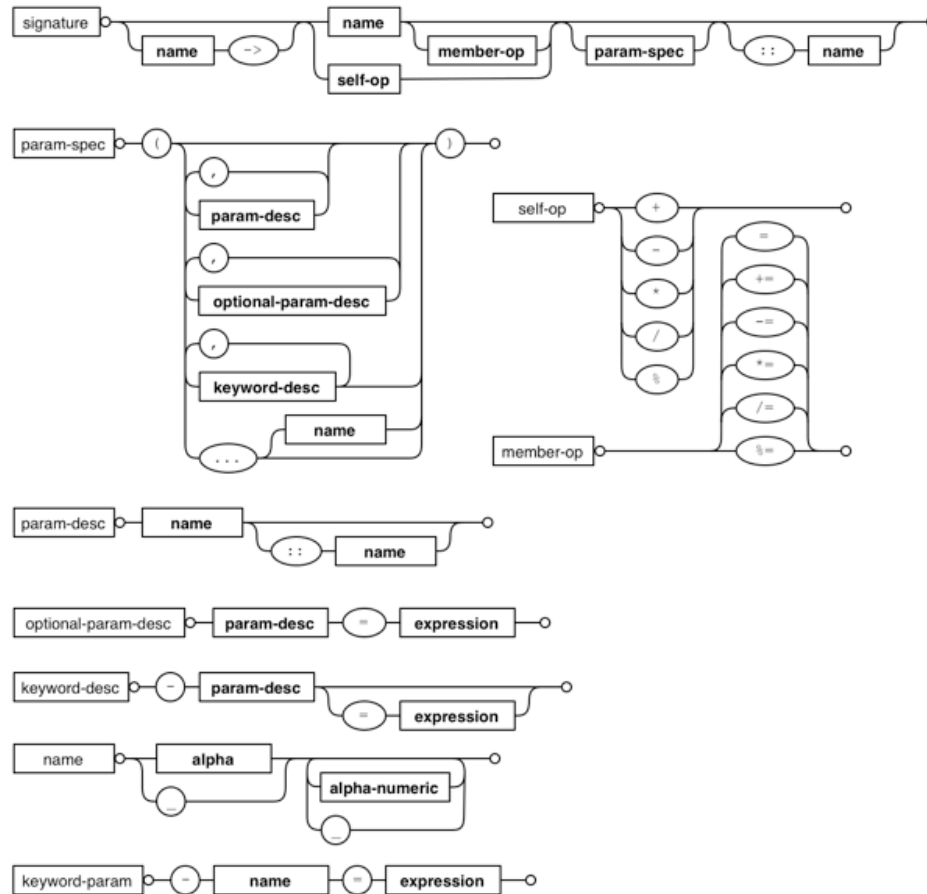
Signatures are also used to denote that the method belongs to a particular data type. This is referred to as the type binding for the signature. A signature with no bound type is referred to as being unbound. All example signatures given up to this point were unbound signatures. A type binding occurs at the beginning of the signature, before the signature's name. It consists of a type name followed by the target operator `->`. The rest of the signature follows that.

```
type_name->method_name(...)
method_name(...)
```

In the above example, the first signature is bound to the type `type_name` while the second signature is unbound. A method using the first signature can not be called except with a target instance of type `type_name`. The second signature can be called at any point, without a target type instance.

Syntax Diagram

What follows are the syntax diagrams for signatures and their related elements.



Defining Methods

Before a method can be utilized, it must first be defined. Defining a method combines together a signature and a method body creating a new method. Defining a method allows it to be called by name from within other methods.

The `define` reserved word is used to define new methods, data types, and traits. When defining a method, the word `define` is followed by a signature, the association operator `=>`, and then an expression which provides the body for the new method.

```
define signature => expression
```

If a method is defined which has a signature identical to an already defined method, the new definition will replace the old and the old definition will no longer be available. Keywords can not be used to uniquely identify a method. A methods which takes, for example, two required parameters and a certain set of keywords will be overwritten by a new method which requires two of the same parameters and an entirely different set of keywords.

Methods Returning Simple Expressions

A simple method definition is shown below. The signature `hello()` describes what and how the method will be called, in this case 'hello' with no parameters. After the association operator `=>` the expression `'Hello, world!'` provides the method's return value. The method below simply returns a string.

```
define hello() => 'Hello, world!'
```

Any single expression, including the ternary-if operator or math expressions can be used as the methods return value. Assignments, local or thread variable declarations, or any other expression known at compilation time to not produce a value may not be used as a method's return value expression.

```
define pi() => math_acos(-1)
define times_twenty(n) => #n * 20
define is_nan(d::decimal) => #d != #d? true | false
```

Code Blocks

Many methods will need to do more than return a single easily calculated value. A method body can be composed of multiple expressions enclosed by a pair of curly-braces `{ ... }`. This type of method body is referred to as a code block.

Code blocks provide the most flexibility when defining methods. They allow a series of expressions to be encapsulated as the implementation of the method. One or more `return` statements may be used to end execution of the method body and to optionally return a value to the caller.

The methods which are used as examples above may be written using code blocks as follows.

```
define pi() => { return math_acos(-1) }
define times_twenty(n) => { return #n * 20 }
define is_nan(d::decimal) => {
  return #d != #d?
    true
    | false
}
```

The expressions within a code block method body are generally formatted so that they each appear on a separate line. Some expressions are terminated by an end-of-line and expressions may be explicitly terminated by using a semi-colon ; at the end of the expression.

The following definition for the hypothetical `strings_combine` uses a series of instructions within the method body to generate the return value for the method.

```
define strings_combine(value::string, with, alsoWith) => {
  local(result = string(#value))

  #result->append(#with->asString)
  #result->append(#alsoWith->asString)

  return #result;
}
```

Syntax Diagram

What follows is the syntax diagram for `define`.



Multiple Dispatch

Multiple dispatch is a technique which permits more than one method body and signature to be defined under a given method name. The various signatures will differ in the number or types of the parameters which they are stated to receive. When the method name is called, the parameters given by the caller (or the lack thereof) will determine which method body will actually be executed. The process of determining which method body to call is referred to as dispatch.

The Dispatch Process

The process of method dispatch first involves taking the name the caller has used and matching it to one or more methods defined under that name. These methods are considered as the set of methods potentially valid for that call. Methods are removed from this set as each parameter value is checked against each valid method's type constraint for that parameter. If the parameter value is acceptable according to this constraint (a lack of a type constraint on a parameter means that any type is valid for that position), then the method remains in the set of valid methods, else it is removed. For each parameter position, methods which accept, at most, fewer than that number of parameters are also removed from the valid set.

In many cases, when the final parameter value is checked there will remain only one valid method. In cases where there are multiple remaining valid methods, the methods are sorted and the top-most method is selected as the method to be executed for that call. The methods are sorted according to how closely related each given parameter value is to each method's stated type constraint for that parameter position, with each subsequent parameter having a lower priority than the previous. Methods with no type constraint for a parameter position will sort lower than methods which do have a type constraint there. For a given position, methods which are valid only because they accept `rest` parameters will sort lower than methods which accept an actual declared parameter there. Additionally, having a required parameter for a position will have a method sort higher than one with an optional parameter.

In the case where the result of the sort leads to two or more equally valid methods, then the call is ambiguous and a failure will be generated. In practice, ambiguous methods are usually

handled when the conflicting method is first defined, leading to the second definition overwriting the first, thus removing the first from future consideration during dispatch.

Keyword parameters are never considered during the method selection process until the end where the single remaining method's keyword parameters (if any) are validated. Two methods can not differentiate themselves based on accepting a different set of keywords. Methods must be distinguished based solely on their required or optional parameters.

Using Multiple Dispatch

Example: Constraints

Multiple dispatch comes into play any time more than one method is defined under a single name. As example, consider the scenario where special diagnostic information needs to be created for a variety of possible types: array, string, bytes and a default "any" type. The `log_object` method is defined multiple times, each accepting a different possible type. Each of the four methods is written to handle only their input value types.

```
define log_object(a::array) => {
  return '[log] array with ' + #a->size + ' elements'
}
define log_object(s::string) => {
  return '[log] string with value "' + #s + '"'
}
define log_object(b::bytes) => {
  return '[log] bytes with hex value 0x' + #b->encodeHex
}
define log_object(any) => {
  return '[log] unhandled object type: ' + #any->type
}

log_object('Hello!')
'\n'
log_object(bytes('ABCD'))
'\n'
log_object(array(1, 2, 3, 4, 5))
'\n'
log_object(pair(1, 2))
```

When the above is executed, the following result is generated.

```
[log] string with value "Hello!"
[log] bytes with hex value 0x41424344
[log] array with 5 elements
[log] unhandled object type: pair
```

Multiple dispatch allows several related methods to be grouped under a single name. This permits method bodies to be more succinct and tailored directly to the input types. This promotes maintainability in a code base, as shorter methods are easier to understand and maintain.

If the above example was instead written to have a single `log_object` method that accepted any value type (we'll call it a mega-method), and within that mega-method, inspected the parameter value type to decide what action to take, then the method would need to be modified each time a new log object type was added. If a log implementation needed to be added for objects of type `pair`, then a new case would need to be placed within that mega-method. What's worse, a user may wish to add their own log implementations for their own object types. If `log_object` were only this single mega-method, then the user would likely have to resort to writing their own set of log methods, falling back to `log_object`'s functionality only for object types that it is known to handle. However, with multiple dispatch, the user may directly add their own `log_object` method with its own unique signature. The new method is incorporated automatically into the system and none of the other methods need to be modified.

```
define log_object(p::pair) => {
  return '[log] pair with: ' + #p->first + ', ' + #p->second
}

log_object('Hello!')
'\n'
log_object(bytes('ABCD'))
'\n'
log_object(array(1, 2, 3, 4, 5))
'\n'
log_object(pair(1, 2))
```

The new log message for inputs of type `pair` is now seen in the result.

```
[log] string with value "Hello!"
[log] bytes with hex value 0x41424344
[log] array with 5 elements
[log] pair with: 1, 2
```

Example: Arity

The number of parameters that a set of methods accepts can be used to determine method dispatch. For example, one method may require a single parameter, while a second method requires two parameters.

```
define log_object(a::array) => {  
  return '[log] array with ' + #a->size + ' elements'  
}  
define log_object(a::array, extra::boolean) => {  
  local(result = log_object(#a))  
  
  #extra?  
    return #result + '. Elements: ' + #a->join(', '  
  
  return #result  
}  
  
log_object(array(1, 2, 3, 4, 5))  
'\n'  
log_object(array(1, 2, 3, 4, 5), true)
```

In the body of the second method, the first method is called to get the initial result string. Then that result is augmented and returned.

```
[log] array with 5 elements  
[log] array with 5 elements. Elements: 1, 2, 3, 4, 5
```